Mississippi Valley State University

CS 341 — Discrete Structures

Practice Set 3: Approved Solutions

Logical Equivalence and the Laws of Logic

Covers Chartrand & Zhang, Section 1.2, pp. 18–24

Status of this set

Ungraded practice. Not submitted; solutions released with it so you can check yourself. Attempt every problem before reading any solution.
Truth tables here have blank cells — fill them in on the page. Everything else goes on your own paper.

Part I: How to Practice This Material

The new skill is rewriting, not evaluating

Until now, every problem asked you to *compute* a truth value. This section asks something different: transform a statement into a different-looking statement that says the same thing. That is closer to algebra than to arithmetic, and it takes different practice.

Which tool for which job

Use a truth table when you are asked whether two statements are equivalent and you do not yet know. The table settles it either way.
Use the laws when you are asked to rewrite or simplify. A chain of named laws is shorter than a table, and it shows *why* the two forms agree rather than merely that they do.
Name the law at every step. "$\sim(P \wedge Q) \equiv (\sim P) \vee (\sim Q)$ by De Morgan" is a complete step. The same line with no name is an assertion, and it earns assertion-level credit.

Proving and disproving cost different amounts

To show $R \equiv S$, every row must agree — all 2^n of them.
To show $R \not\equiv S$, produce **one** row where they differ. You do not need the rest of the table, though building it is usually the fastest way to find that row.
Several problems below are disproofs. Do not do more work than the job requires, but do state the differing row explicitly — "they're not equivalent" without a witness is not an answer.

Four mistakes to expect

1. **Negating the pieces and leaving the connective alone.** The most expensive error in this section, and the one De Morgan exists to prevent.
2. **Dropping parentheses across different connectives.** $P \wedge Q \wedge R$ is fine. $P \wedge (Q \vee R)$ is not the same as $(P \wedge Q) \vee R$.
3. **Applying a law you have not checked.** A pattern that works for $\wedge$ and $\vee$ may fail for $\oplus$. Two problems below turn on exactly that.
4. **Skipping the law names in a simplification.** If a grader cannot tell which step you took, the step did not happen.

Part II: Practice Problems

Section A — Prove or disprove

1. Determine whether $\sim(P \vee Q)$ and $(\sim P) \vee (\sim Q)$ are logically equivalent. If they are not, name every row where they differ.

P	Q	$\sim(P \vee Q)$	$\sim P$	$(\sim P) \vee (\sim Q)$
T	T	F	F	F
T	F	F	F	T
F	T	F	T	T
F	F	T	T	T

Approved solution

Not equivalent. They differ in two rows: P true with Q false, and P false with Q true.
Why this is worth doing. De Morgan says $\sim(P \vee Q) \equiv (\sim P) \wedge (\sim Q)$ — with an *and*. Changing that one connective to *or* breaks the law completely. Students who remember De Morgan as "push the negation inside" without tracking what happens to the connective produce exactly this wrong statement.
Note also how little work a disproof needs. Once you reach row two and see F against T, you are finished. The remaining rows are optional.

2. Is $\oplus$ associative? That is, determine whether $P \oplus (Q \oplus R) \equiv (P \oplus Q) \oplus R$.

P	Q	R	$Q \oplus R$	$P \oplus (Q \oplus R)$	$P \oplus Q$	$(P \oplus Q) \oplus R$
T	T	T	F	T	F	T
T	T	F	T	F	F	F
T	F	T	T	F	T	F
T	F	F	F	T	T	T
F	T	T	F	F	T	F
F	T	F	T	T	T	T
F	F	T	T	T	F	T
F	F	F	F	F	F	F

Approved solution

Yes — $\oplus$ is associative. The two final columns agree in all eight rows.
Look at the pattern in either column: the result is true exactly when an *odd number* of P, Q, R are true. That is not a coincidence, and it is why $\oplus$ is the operation behind parity bits: chaining exclusive-ors across a block of bits reports whether the number of ones is odd, and the grouping cannot affect the answer.

3. Determine whether $P \vee (Q \oplus R) \equiv (P \vee Q) \oplus (P \vee R)$. If not, name every row where they differ.

P	Q	R	$Q \oplus R$	$P \vee (Q \oplus R)$	$P \vee Q$	$(P \vee Q) \oplus (P \vee R)$
T	T	T	F	T	T	F
T	T	F	T	T	T	F
T	F	T	T	T	T	F
T	F	F	F	T	T	F
F	T	T	F	F	T	F
F	T	F	T	T	T	T
F	F	T	T	T	F	T
F	F	F	F	F	F	F

Approved solution

Not equivalent. They differ in all four rows where P is true: TTT, TTF, TFT, TFF.
The reason is visible once you look. When P is true, $P \vee Q$ and $P \vee R$ are both true no matter what Q and R are — so their exclusive or is false, every time. Meanwhile the left side is true whenever P is. The two sides are as far apart as they can be on that half of the table.
The lesson. $\vee$ distributes over $\wedge$, and $\wedge$ distributes over $\vee$. It is tempting to conclude that connectives simply distribute over one another. They do not. Every distributive claim needs checking, and this one is false.

Section B — Simplify using the laws

For each, produce a chain of equivalences ending in a statement with *no outer negation*. Name the law used at every step.

4. $\sim\big((\sim P) \vee Q\big)$

Approved solution

$$\begin{aligned} \sim\big((\sim P) \vee Q\big) &\equiv \big(\sim(\sim P)\big) \wedge (\sim Q) && \text{De Morgan (Thm 1.24)} \\ &\equiv P \wedge (\sim Q) && \text{Double negation (Thm 1.28)} \end{aligned}$$

In words: P holds and Q fails.
Two steps, two named laws. A student who writes only the final line has the right answer and has not shown the work the problem asked for.

5. $\sim\big((\sim P) \wedge (\sim Q)\big)$

Approved solution

$$\begin{aligned} \sim\big((\sim P) \wedge (\sim Q)\big) &\equiv \big(\sim(\sim P)\big) \vee \big(\sim(\sim Q)\big) && \text{De Morgan (Thm 1.24)} \\ &\equiv P \vee Q && \text{Double negation, twice (Thm 1.28)} \end{aligned}$$

In words: the negation of "neither P nor Q" is "P or Q" — which is what common sense already said. The laws agree with intuition here; the point of doing it symbolically is that they will keep agreeing when intuition runs out.

6. $\sim\big(\sim(P \wedge Q)\big)$

Approved solution

$$\sim\big(\sim(P \wedge Q)\big) \equiv P \wedge Q \qquad \text{Double negation (Thm 1.28)}$$

One step. The temptation is to apply De Morgan to the inner negation first, grind through $(\sim P) \vee (\sim Q)$, negate that, and arrive back at $P \wedge Q$ several lines later. That route is correct but wasteful. **Look at the whole expression before you start pushing symbols around.** That habit matters more later, when the expressions get long.

Section C — Negation in words

7. Write the negation of each statement, phrased the way a person would say it.

(a) The cache is warm and the query is indexed.

(b) Either n is prime or n is composite.

(c) $0 < x < 1$

(d) The user is neither an administrator nor the owner of the file.

(e) The list is sorted and contains no duplicates.

Approved solution

(a) The cache is not warm **or** the query is not indexed.
(b) n is not prime **and** n is not composite. (Worth noticing: this is actually possible — $n = 1$ is neither.)
(c) $x \leq 0$ **or** $x \geq 1$.
The notation $0 < x < 1$ hides a conjunction: it means $x > 0$ *and* $x < 1$. Negating it flips the connective to *or* and flips both inequalities, each of which picks up its equality case. Writing $0 > x > 1$ describes no number at all.
(d) The user is an administrator **or** the owner of the file. "Neither… nor" already means "not this and not that," so its negation is a disjunction.
(e) The list is not sorted, **or** it contains at least one duplicate.
Note the second half: the negation of "contains no duplicates" is "contains at least one duplicate," not "contains only duplicates." That is a quantifier negation, and it becomes formal in Section 3.1.

All five are De Morgan. If your answer to any of them kept the original connective, fix that before the next assignment is due.

Section D — Rewriting conditions in code

8. A reviewer flags this condition as needlessly hard to read:

```
if not (not logged_in or not has_permission):
    grant_access()
```

Rewrite the condition in its simplest equivalent form, naming the laws you use. Then say in one sentence what the condition means.

Approved solution

Let L: the user is logged in, and H: the user has permission.

$$\begin{aligned} \sim\big((\sim L) \vee (\sim H)\big) &\equiv \big(\sim(\sim L)\big) \wedge \big(\sim(\sim H)\big) && \text{De Morgan (Thm 1.24)} \\ &\equiv L \wedge H && \text{Double negation (Thm 1.28)} \end{aligned}$$

```
if logged_in and has_permission:
    grant_access()
```

In words: access is granted exactly when the user is both logged in and permitted.
The original was correct. It was just written inside out, and three negations were doing the work of none. This is the everyday use of these laws — not proving theorems, but making a condition say plainly what it means.

9. For each pair of conditions below, the two operands are logically equivalent under the commutative law. Say whether the two operands can safely be *swapped* in the code, and explain what distinguishes the two cases.

```
(a) if index < len(items) and items[index] > 0:

(b) if x > 0 and y > 0:
```

Approved solution

(a) Not safe to swap. Written as `items[index] > 0 and index < len(items)`, the program indexes the list *before* checking that the index is in range, raising an `IndexError` whenever `index` is too large. The first version never evaluates the second operand in that case, because `and` stops as soon as the left side is false.
(b) Safe to swap. Both operands are simple comparisons. Evaluating either one first is harmless, and the result is the same.
What distinguishes them: whether evaluating an operand can do something other than produce a truth value. In (a) the second operand can raise an exception; its evaluation is *guarded* by the first. In (b) neither operand can fail or change anything.
The commutative law is true in both cases — $P \wedge Q \equiv Q \wedge P$ always. What differs is whether the program is only computing a truth value, or also relying on the order in which the parts get evaluated. Logic describes the first thing and says nothing about the second.

Section E — Writing

10. In five to eight complete sentences, explain how both of the following can be true at once:

- $P \wedge Q \equiv Q \wedge P$ — the commutative law, proved by truth table.
- Swapping the two sides of an `and` in a program can crash it.

Your explanation must *not* claim that the commutative law is wrong or that it has exceptions. Refer to Problem 9 specifically.

Approved solution

Model answer. The commutative law is a statement about truth values, and it holds without exception: given the truth values of P and Q, the compound statements $P \wedge Q$ and $Q \wedge P$ receive the same truth value in every row of the table. But `and` in a program does two separate jobs. It computes a truth value, and it also determines whether the second operand is evaluated at all — if the left side is false, the right side is skipped. In Problem 9(a), that second job is what keeps the program alive: `index < len(items)` guards `items[index]`, so the indexing never happens when the index is out of range. Swapping the operands removes the guard, and the program raises an exception before any truth value is produced. Nothing about the commutative law failed here; the crash happens in a part of the program's behavior that truth values do not describe. The lesson is that logical equivalence tells you two conditions *mean* the same thing, not that they are interchangeable in every context.

What earns full credit: the answer must locate the difference in what a truth table *describes*, not in an exception to the law. Any answer of the form "the law is true in math but not in programming" has missed it — the law is true in programming too. It simply does not say what the student wants it to say.
Common weak answer: "because of short-circuit evaluation." That names the mechanism without explaining why it does not contradict the law. Award roughly half.

Section F — Challenge (optional)

11. In Problem 3 you found that $\vee$ does *not* distribute over $\oplus$. Determine whether $\wedge$ does: is $P \wedge (Q \oplus R) \equiv (P \wedge Q) \oplus (P \wedge R)$?

Approved solution

Yes, this one holds. Both sides are true exactly in rows TTF and TFT — that is, when P is true and exactly one of Q, R is true.

P	Q	R	$Q \oplus R$	$P \wedge (Q \oplus R)$	$P \wedge Q$	$(P \wedge Q) \oplus (P \wedge R)$
T	T	T	F	**F**	T	**F**
T	T	F	T	**T**	T	**T**
T	F	T	T	**T**	F	**T**
T	F	F	F	**F**	F	**F**
F	T	T	F	**F**	F	**F**
F	T	F	T	**F**	F	**F**
F	F	T	T	**F**	F	**F**
F	F	F	F	**F**	F	**F**

The pair is the point. $\wedge$ distributes over $\oplus$; $\vee$ does not. Nothing about the shape of the two claims predicts which way each goes — only the tables do. When you meet an unfamiliar identity, that is the position you are in.

12. De Morgan's Laws are stated for two statements. Verify the three-statement version: $\sim(P \vee Q \vee R) \equiv (\sim P) \wedge (\sim Q) \wedge (\sim R)$. Then say in one sentence what the general pattern must be for n statements.

Approved solution

The equivalence holds. Both sides are false in the first seven rows and true only in the last, where P, Q, and R are all false.

The reasoning is short: $P \vee Q \vee R$ is false exactly when all three are false, so its negation is true exactly then — which is precisely what $(\sim P) \wedge (\sim Q) \wedge (\sim R)$ says.

The general pattern: negating a disjunction of any number of statements gives the conjunction of all their negations, and negating a conjunction gives the disjunction of all their negations. In words — "none of them" is the negation of "at least one of them," and "at least one fails" is the negation of "all of them."

That second phrasing is worth holding onto. In Section 3.1 the same idea reappears as the rule for negating quantified statements, where "all" and "some" trade places.

Self-check rubric

Score	Level	Description
4	Complete	Correct result, every law named, conclusion stated in a sentence.
3	Sound but thin	Correct result; a law unnamed or a step compressed.
2	Right answer, weak support	Correct result with no named laws, or a disproof with no witness row.
1	Answer only	A bare final expression or "not equivalent" with nothing else.
0	Incorrect	Wrong result, or a law applied that does not hold.

Instructor notes

- **Nothing here touches Homework 2 Problem 3.** The $\vee$-over-$\wedge$ distributive law stays open until August 28. Problems 3 and 11 explore distribution over $\oplus$ instead, which is untouched territory and makes the "check every distributive claim" point more forcefully than the $\wedge/\vee$ pair could.
- **Problems 3 and 11 are a matched pair** and should be discussed together. $\wedge$ distributes over $\oplus$; $\vee$ does not. Nothing in the surface form predicts which. This is the strongest argument available for why students may not pattern-match their way through identities.
- **Problem 7(c)** rehearses the hidden conjunction in $0 < x < 1$, which is the same pattern as Homework 2 Problem 2(c). That transfer is intended — it is the highest value negation pattern in the section and students need it more than once.
- **Problems 9 and 10** are the pair that separates strong students. Anyone who writes "the commutative law doesn't apply to code" has the wrong resolution; the law does apply, it simply does not describe evaluation order.
- **Problem 6** rewards reading before manipulating. Watch for students who apply De Morgan twice and arrive back where they started — correct, and worth a word about looking at the whole expression first.
- **Problem 12** plants the quantifier negation rule two weeks early. Recall it by name when you reach Section 3.1.